Part 4: Results and Methods

Data and scope

We study PJM day-ahead LMP and (separately) load forecasts joined to NOAA ISD hourly weather at station locations mapped to each pricing node. The current correlation export contains 1974 node–year–variable records (weather vs. LMP, Pearson r) across 76 nodes and 24 balancing-area zones; calendar years 2021–2025 appear in the series. Period tags in code treat `earlyAI` as the 2021 slice and `recentAI` as 2022–2025, so year-to-year trend plots partially confound calendar year with the period definition.

Methods

Weather merge and features. For each node and year, PJM hourly records were aligned to UTC and merged to NOAA "global hourly" fields using nearest-time matching (`merge_asof`, 30 minute tolerance). Sentinels were nulled; temperature, dew point, pressure, wind, and precipitation were converted to physical units. Cooling and heating degree hours (base 18 °C) and relative humidity (Magnus form) were computed in `scripts/build_analysis_exports.py` and `scripts/train_rq2_models.py`.

RQ1 (associational). For each node, year, weather variable, and target (day-ahead LMP or load), we computed Pearson correlation between the merged weather series and the target. Rows carry a DC vs. non-DC bucket derived from explainable node scoring (`scripts/export_pjm_metadata.py`) propagated into `data/exports/correlations_by_region_period.json`.

RQ2 (predictive). We featurized merged hourly panels per node–year (weather signals plus calendar indicators where implemented), imposed a chronological evaluation split with held-out test years starting at 2024, median-imputed missing values, standardized inputs for linear learners, and trained six regressors per configuration: ordinary least squares, Ridge (α=1), Lasso (α=0.001), RandomForest (90 trees), GradientBoosting, and ExtraTrees (100 trees). Metrics were computed on the held-out portion (RMSE, MAE, R^2); weather-only nRMSE = RMSE/$|\bar{y}|$ when the held-out target mean $\bar{y}$ is available. The training entry point is `python scripts/train_rq2_models.py` (flags for year range, node cap, and data volume). Results are written to `data/exports/model_performance.json` and consumed by the React app after `npm run sync:data`.

Between-group summaries. `scripts/build_hypothesis_exports.py` aggregates node-level absolute correlations for RQ1 and weather-only nRMSE rows for RQ2; it computes bootstrap 95% confidence intervals for the DC-minus-non-DC mean contrast (when minimum group sizes are met), two-sided permutation p-values (3000 draws, seed 42), Cohen's d, and marks descriptive-only outputs when gates fail.

Figures (trends vs. bucket contrasts). Figures 1–2 regenerate from `correlations_by_region_period.json` via `docs/generate_report_figures.py`. DC-aware distribution and bootstrap panels (Figs. 3–6) come from `docs/generate_rq_research_figures.py`, using `nodes.json`, correlations, `model_performance.json`, and `hypothesis_tests.json`.

Results

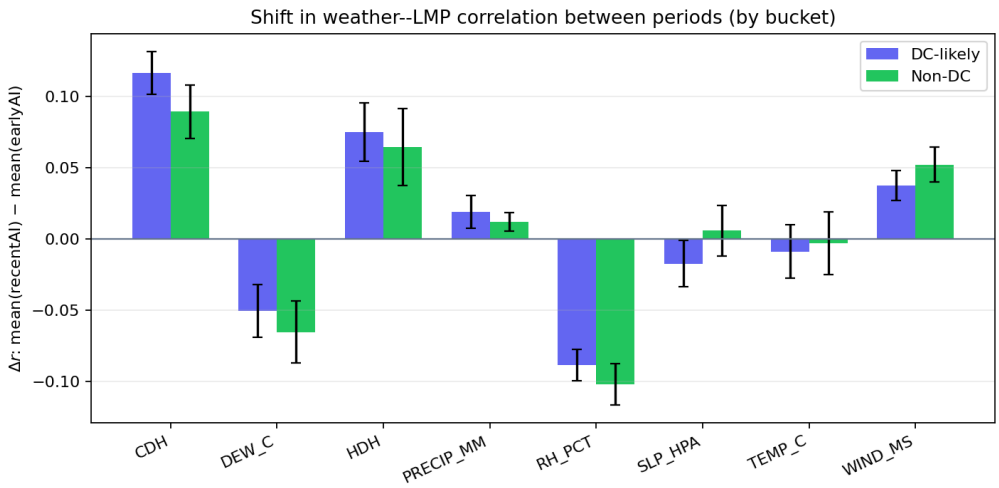

Figure 1: Fig. 1: Mean shift in weather–LMP correlation between `recentAI` (2022–2025) and `earlyAI` (2021), by DC-likelihood bucket and weather variable; error bars reflect independent SEM propagation across periods.

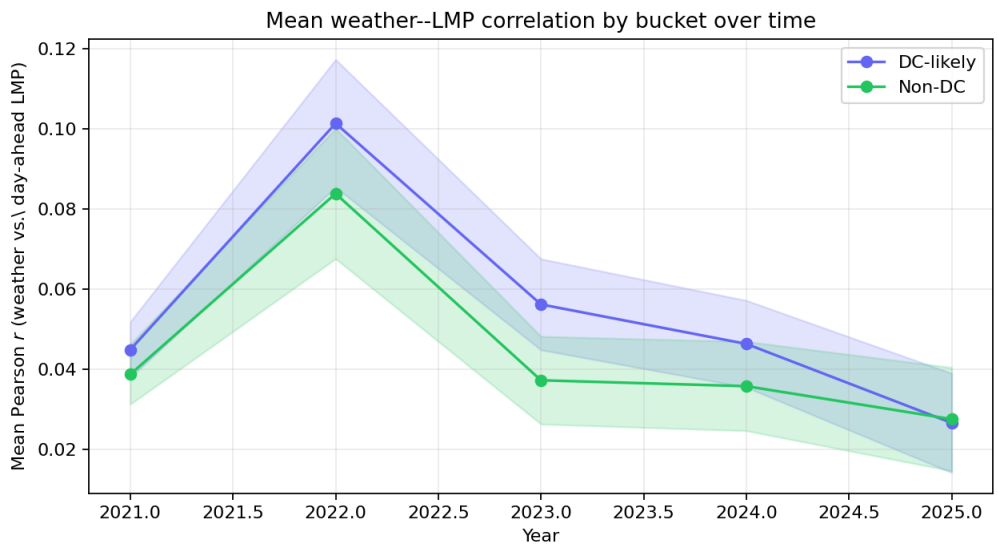

Figure 2: Fig. 2: Mean Pearson r (weather vs. LMP) by calendar year and bucket; shaded bands are ± 1 SEM within each year$\times$bucket cell.

Figures 1–2 summarize variable-level associational contrasts over time (RQ1 context); they compress many nodes into bucket means.

RQ1 and RQ2: distributions and mean-difference uncertainty. Figures 3–6 emphasize spread, n, and uncertainty rather than bar-chart means alone. Distribution panels are *descriptive*;

CI panels repeat the locked inferential summaries from Table 1 (bootstrap CI for DC − non-DC mean contrast; permutation p in the table).

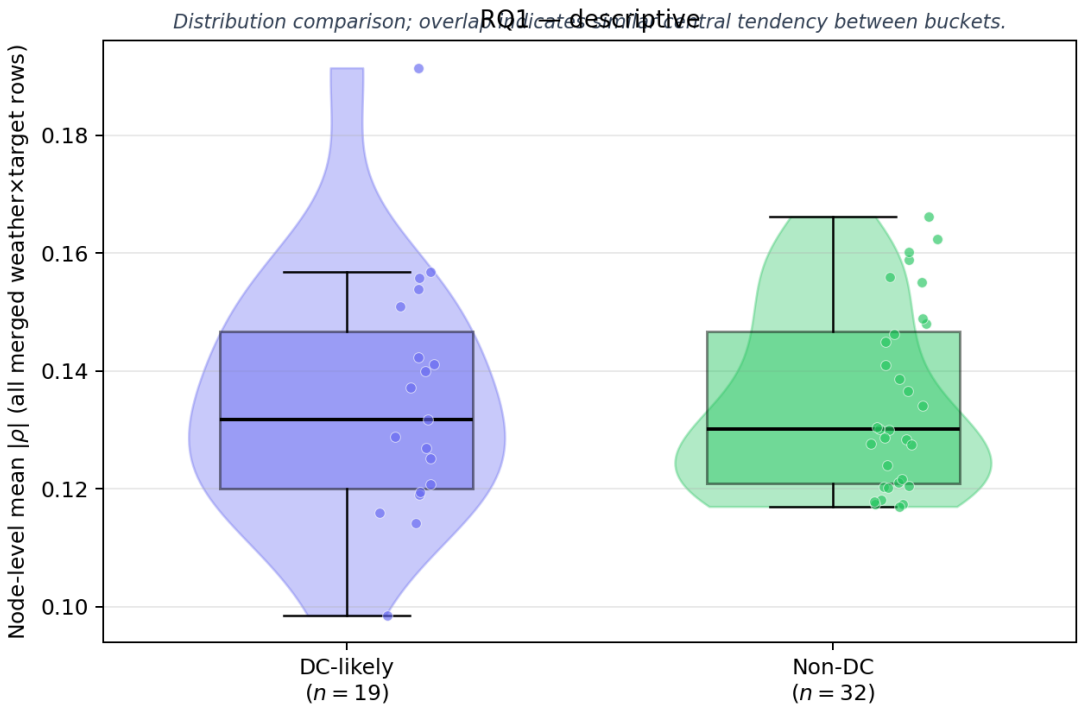

Figure 3: **Descriptive (RQ1).** Node-level mean absolute Pearson correlation (all merged weather×target rows per node), by DC-likelihood bucket. Violin shading, box quartiles, and jittered points show overlap and tails; axis labels include group n.

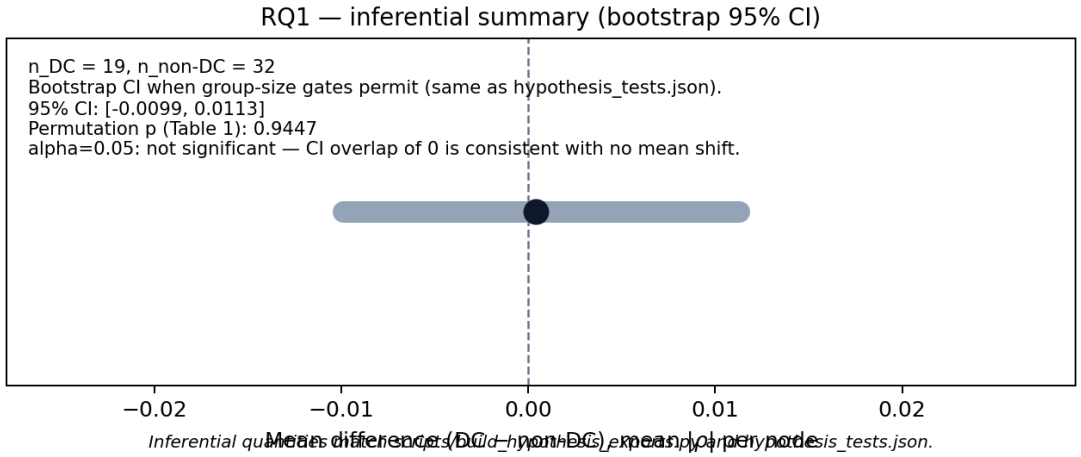

Figure 4: **Inferential (RQ1).** Bootstrap 95% CI for the difference of *group means* of node-level mean $|\rho|$ (DC − non-DC), aligned with `hypothesis_tests.json`. Interval crossing 0 is consistent with no detectable mean shift at this precision; significance follows Table 1, not the graphic alone.

For RQ1, Figs. 3–4 support the same story as Table 1: distributions largely overlap and the

inferential CI for the mean difference straddles zero ($p = 0.945$).

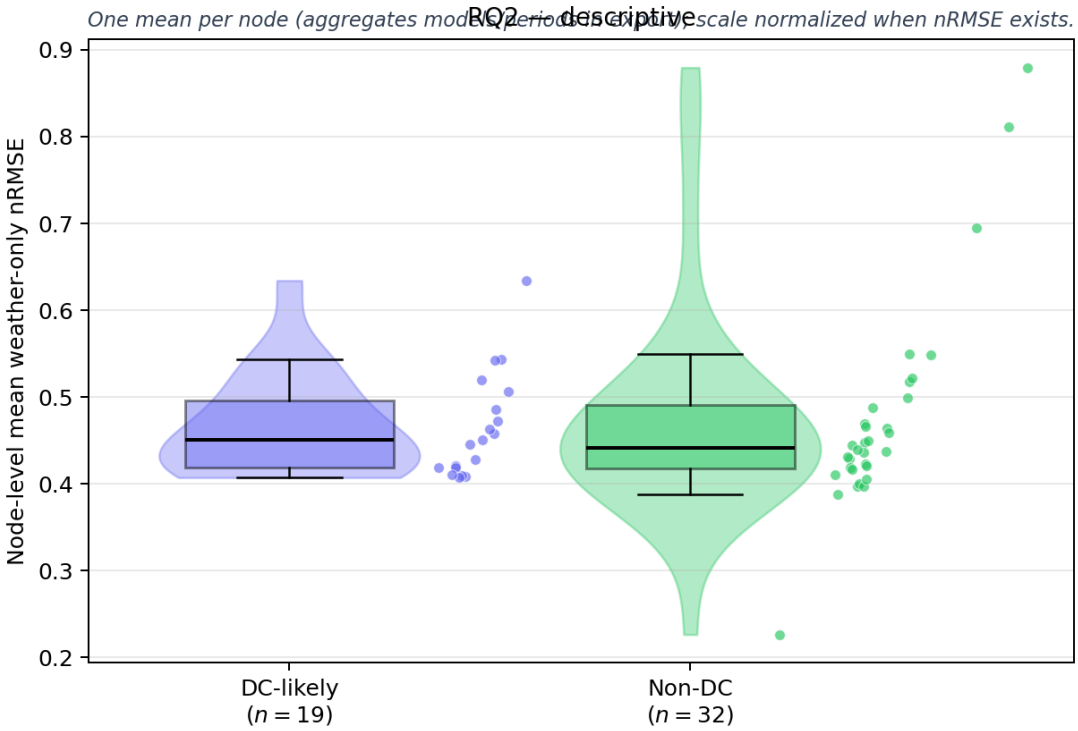

Figure 5: **Descriptive (RQ2).** Node-level mean weather-only error (per-node average across exported evaluation rows; nRMSE when available so scale is comparable across nodes). High dispersion reflects model and period heterogeneity within buckets.

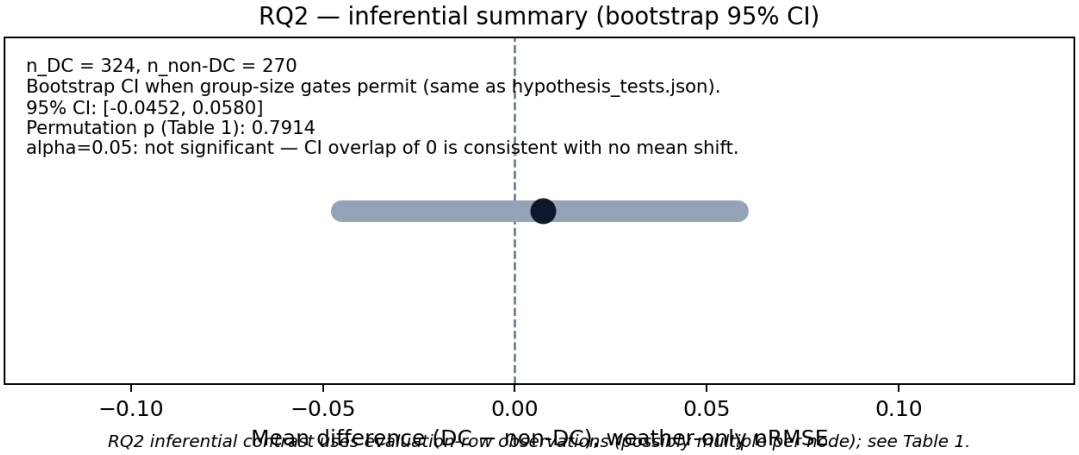

Figure 6: **Inferential (RQ2).** Bootstrap 95% CI for DC − non-DC mean weather-only nRMSE using *evaluation rows* (possibly multiple per node), as in the hypothesis export. Descriptive Fig. 5 uses node-level means for clarity; interpret inferential claims from Table 1 and this panel.

For RQ2, Fig. 5 shows substantial within-bucket spread; Fig. 6 and Table 1 show a small mean contrast with CI spanning zero ($p = 0.791$), so we do not claim a reliable accuracy gap between

buckets under this specification.

Table 1: Hypothesis-export contrasts at $\alpha = 0.05$ (from `hypothesis_tests.json`). CI = bootstrap mean difference DC − non-DC.

RQ	Metric	n DC / non-DC	Means (DC; non-DC)	CI / p / Cohen's d
1	Mean $\|r\|$ per node	19; 32	0.135; 0.135	[−0.010, 0.011]; p=0.945; d=0.022
2	Weather-only nRMSE (rows)	324; 270	0.465; 0.457	[−0.045, 0.058]; p=0.791; d=0.023

Table 1 matches Figs. 4 and 6: neither mean contrast is significant at $\alpha = 0.05$ (RQ1 $p = 0.945$, RQ2 $p = 0.791$); Cohen's d is negligible in both cases.

The React dashboard (`OverviewPage`) mirrors these panels (box + jitter + CI cards; Models page: filters by target and learner). Inferential captions follow Table 1.

Reproducibility

- **Repository:** `https://github.com/jaiparmar940/energy-weather-app`
- **Trend figures (Fig. 1–2):** `pythondocs/generate_report_figures.py`
- **RQ distribution / CI figures (Fig. 3–6):** `pythondocs/generate_rq_research_figures.py`
- **Model and hypothesis exports:** `pythonscripts/train_rq2_models.py`; then `pythonscripts/build_hypothesis_exports.py`; then `npmrunsync:data`.